HPE Networking Assistant — Test Plan

End-to-end acceptance test plan covering the **installation process** and **every tool and visualizer**. Written to be executed on a single Windows PC acting as the test client.

- **Product version under test:** 1.9.0
- **Repository:** https://github.com/hpe-networking-lab/hpe-networking-assistant
- **Two test layers:**
  1. **Automated** — the `pytest` suite (65 tests) validates all tool logic with a mocked Mist API; no token or network required.
  2. **Manual acceptance** — live tests against a real Mist tenant through Claude Desktop, below.
  3. **Companion dashboards** — two live Cowork artifacts (Section 9a), separate from the packaged `.dxt`.

Mark each case **PASS / FAIL** in the result column and capture notes/screenshots for failures.

---

1. Test environment

Item	Value for this run
Test client	This Windows PC
OS	Windows 10/11
Python	3.10+ (`python --version`)
Claude Desktop	Latest
Mist tenant	_Use a **lab/non-production** org for write tests_
Mist token role (read tests)	Observer / read
Mist token role (write tests)	Network Admin / write
Region	Auto-detected

Safety

- **Read-only tools are safe in production.** Run them anywhere.
- **Write-mode tests (Section 9) change device state/config** (rename, reboot, LED, claim/assign). Run them **only against a lab org** and only on devices you may disrupt.

---

2. Prerequisites checklist

#	Check	Expected	Result
P1	`python --version`	3.10 or newer	
P2	Claude Desktop installed and opens	Launches	
P3	Mist API token created (My Account → API Token)	Token copied	
P4	(Optional) Git installed for the dev/automated path	`git --version` works	

3. Automated test suite (developer path)

Run from a clone of the repo. Validates all tools/visualizers offline.

#	Step	Command	Expected	Result
A1	Clone + install	`pip install -e ".[dev]"`	Installs with no errors	
A2	Run tests	`pytest -q`	**82 passed**	
A3	Byte-compile	`python -m compileall src server`	No errors	
A4	Validate manifest	`python -c "import json;json.load(open('manifest.json'))"`	No error	

Coverage map (automated):

Area	Test file
Region normalization + detection	`test_regions.py`, `test_discovery.py`
Mist client (auth, pagination, writes, search)	`test_mist_client.py`, `test_write_mode.py`, `test_client_trace.py`, `test_access_assurance.py`
MCP server / dispatch / tool registry	`test_server.py`
Onboarding & status	`test_onboarding.py`, `test_status.py`
Validation framework	`test_validation.py`
Reports	`test_reports.py`
NAC visualizer	`test_nac_visualizer.py`

4. Installation tests (TC-INST)

#	Objective	Steps	Expected	Result
INST-1	Download artifact	From the latest release, download `hpe-networking-assistant-<ver>.dxt`	File downloads	
INST-2	Verify checksum (optional)	Compare SHA-256 against `SHA256SUMS.txt`	Matches	
INST-3	Install extension	Claude Desktop → Settings → Extensions → drag in the `.dxt`	Extension details + config form appear	
INST-4	Config form is token-only	Inspect the form	Only **Mist API Token** (required) and **Enable write operations** (off) are shown — no region/org fields	
INST-5	Enter token + enable	Paste token, leave write off, Install/Enable	Extension enables without error	

#	Objective	Steps	Expected	Result
INST-6	Token stored securely	—	Token saved to OS keychain (not shown again in plain text)	
INST-7	Time budget	From clicking install to first successful query	**Under 10 minutes**	

5. Onboarding tests (TC-ONB)

#	Objective	Prompt / action	Expected	Result
ONB-1	First-run wizard	New chat: **"Set me up."**	Claude calls `start_setup`; auto-detects region, discovers orgs/sites, runs validation, returns **READY FOR USE**	
ONB-2	Region auto-detected	Observe report	Correct region shown; you never entered it	
ONB-3	No IDs required	Observe	You are never asked for Org ID / Site ID / API endpoint	
ONB-4	Multi-org prompt	(If token sees >1 org)	Claude lists orgs **by name** and asks which to use; answering by name completes setup	
ONB-5	Persistence	Restart Claude Desktop, ask "what's my status?"	Region/org remembered (no re-probe needed)	

6. Read-only tool tests (TC-RO)

Use natural-language prompts; verify Claude calls the right tool and returns sensible data.

#	Tool	Prompt	Expected	Result
RO-1	`get_status`	"Am I in read-only or read-write mode?"	Reports read-only, region, org, account	
RO-2	`get_organizations`	"What organizations can I access?"	Lists org name(s)	
RO-3	`get_sites`	"List my sites."	Site names + locations	
RO-4	`get_access_points`	"Show all APs in my organization." (success-criteria query)	AP list with online/offline counts	
RO-5	`get_access_points` (site)	"Show APs at <site name>."	APs filtered to that site	
RO-6	`get_switches`	"List my switches."	Switch list with status	
RO-7	`get_clients`	"How many clients are connected at <site>?"	Client count/list	
RO-8	`get_offline_access_points`	"Which access points are offline right now?"	Offline AP list (matches portal)	
RO-9	Cross-check	Compare RO-4/RO-8 counts to the Mist portal	Numbers match	
RO-10	`get_wired_clients`	"List wired clients / what's on switch port X?"	Wired clients with switch MAC, port, VLAN, IP, vendor	

#	Tool	Prompt	Expected	Result
RO-11	`get_marvis_actions`	"What's wrong with my network?"	Marvis suggested actions + fixes, per-category counts; `status=open` filters to active	
RO-12	`get_alarms`	"Show critical alarms in the last day."	Alarms with per-severity/per-type counts	
RO-13	`get_sle`	"How's the user experience by site?"	Per-site SLE metrics as percentages	
RO-14	`get_switch_ports`	"Show ports on switch <mac> / down ports"	Port link/speed/PoE/neighbor; filters by switch/site/up	
RO-15	`export_org_config`	"Back up my org config to a file."	JSON bundle (org + sites/networks/templates/WLANs/NAC/…) with per-type counts; saved as .json	

---

7. Report tests (TC-RPT)

#	Tool	Prompt	Expected	Result
RPT-1	`generate_health_report`	"Generate a network health report and save it as a file."	Markdown report (totals, offline APs, per-site) saved as `.md`	
RPT-2	Health accuracy	Open the report	Counts match RO-4/RO-6/RO-8	
RPT-3	Skip clients	"Generate a health report without the client count."	Report notes clients "not collected" (faster)	
RPT-4	`generate_inventory_report`	"Create an inventory report of all devices."	Markdown table: every AP+switch with model/serial/MAC/site/status/version	
RPT-5	Convert	"Convert that report to PDF."	Claude produces a PDF from the Markdown	

---

8. Client trace & Access Assurance tests (TC-CT / TC-AA)

> Pick a real client MAC/hostname from RO-7 for these.

#	Tool	Prompt	Expected	Result
CT-1	`find_client` (mac)	"Find the client with MAC <mac>."	Site, AP, SSID, IP, band, RSSI, last seen	
CT-2	`find_client` (hostname)	"Where is hostname <name>?"	Matching client(s) located	
CT-3	`trace_client`	"Trace client <mac> over the last hour."	Event timeline, per-type counts, highlighted failures	
CT-4	No-input guard	"Find a client." (no mac/hostname)	Friendly error asking for mac or hostname	
AA-1	`get_nac_clients`	"List NAC clients from the last day."	Authenticated NAC clients (user, type, auth type, SSID, VLAN, rule, status) — _empty if Access Assurance not configured_	
AA-2	Filter	"Show only EAP-TLS NAC clients."	Filtered list	
AA-3	`troubleshoot_authentication`	"Troubleshoot authentication failures in the last 24 hours."	Event timeline, type counts, failure highlights	

#	Tool	Prompt	Expected	Result
AA-4	Focus by client (MAC)	"Why is <mac> failing authentication?"	Events scoped to that MAC	
AA-5	Focus by user (`user`)	"Why is bob@corp failing 802.1X?"	Events scoped to that username/cert CN (free-text), failures highlighted	

> **Note:** AA tests require a tenant with **Access Assurance (NAC)** enabled. If your tenant has no NAC, expect empty results (not an error). Confirm the NAC events endpoint path against your tenant (see Release Notes 1.6.0).

---

9. NAC dashboard test (TC-NAC)

#	Tool	Prompt	Expected	Result
NAC-1	`generate_nac_dashboard`	"Build a NAC dashboard and save it as HTML."	Self-contained `.html` produced	
NAC-2	Open offline	Double-click the `.html`	Opens in browser; cards + bar charts render with **no internet**	
NAC-3	Content	Inspect	Cards (clients, events, failures, success rate) + charts for auth types, client types, status, event types, top failing users/rules	
NAC-4	Empty-state	(No-NAC tenant)	Renders with zeros / "No data" rather than erroring	

9a. Companion Cowork dashboards (TC-CD)

> Live, connector-backed dashboards in Claude Cowork, separate from the packaged extension. They require the Mist connector to be connected in Cowork.

#	Objective	Action	Expected	Result
CD-1	Open network/NAC dashboard	Open the "Mist Network & Access Assurance" artifact	Device-status doughnut + cards load from live data	
CD-2	Auto-refresh	Leave it open ~1 min	Updates on its own (~30s cadence); "last updated" time advances; pulsing live dot	
CD-3	NAC empty-state	(No-NAC tenant)	Shows "Access Assurance may not be configured" rather than erroring	
CD-4	Open NAC Auth Debugger	Open the "NAC Auth Debugger" artifact	Loads with input + time-window controls	
CD-5	Debug by username	Type a username, choose a window, click Debug	Pulls that identity's NAC events; pass/fail counts; timeline	
CD-6	Debug by MAC	Enter a 12-hex MAC	Auto-routes to MAC filter; events returned	
CD-7	Failure decode	(Identity with a failure)	"Most recent failure" card shows reason, matched rule, auth type, NAS	

#	Objective	Action	Expected	Result
CD-8	Explain hand-off	Click "Ask Claude to explain and suggest a fix"	Failure detail is sent to chat as a prompt	

---

10. Write-mode tests (TC-WR) — LAB ORG ONLY

> These change device state/config. Use a lab org and a **write-capable token**.

#	Objective	Prompt / action	Expected	Result
WR-1	Enable write mode	Settings → toggle **Enable write operations** on; restart prompt	Extension restarts	
WR-2	Write tools appear	"What can you do now?" / list tools	`rename_device`, `reboot_device`, `locate_device`, `claim_devices`, `assign_devices_to_site` now available	
WR-3	Write-readiness	"Check my setup." (`get_status`)	If token is read-only → reports it + guidance; if write-capable → confirms write access	
WR-4	Confirmation gate	"Rename AP <mac> to TEST-AP."	Claude asks to confirm; nothing changes until you confirm	
WR-5	Rename	Confirm WR-4	Device renamed (verify in portal); revert afterward	
WR-6	LED locate	"Blink the locate LED on <mac>." → confirm	LED toggles (verify physically/portal)	
WR-7	Reboot (disruptive)	"Reboot <mac>." → confirm	Device restarts (lab device only)	
WR-8	Read-only token blocked	With a read-only token + write on, attempt a write	Mist returns permission error; guidance shown	
WR-9	Disable write	Toggle off; restart	Write tools no longer offered	

---

11. V